نزدیک پردازنده‌ای باشند که برای اجرا زمانبندی می‌شود، در این صورت توفیق بیشتر کَش و تقلیل زمان‌های دسترسی حافظه قابل حصول است.

اوضاع با آمدن نخ‌ها از این هم پیچیده‌تر می‌شود. برای مثال، یک پردازه با نخ‌های اجرایی فراوان به جایی می‌رسد که نخ‌های آن بر روی بردهای چندین سیستم متفاوت زمانبندی می‌شوند. در این حالت، تخصیص حافظه بایستی چگونه صورت بگیرد؟

همان‌گونه که در بخش ۵-۷-۱ بحث کردیم، لینوکس این وضعیت را با اجبار هسته به شناسایی یک سلسله‌مراتبی از دامنه‌های زمان‌بندی مدیریت می‌کند. زمان‌بند CFS لینوکس به نخ‌ها اجازه نمی‌دهد تا در میان دامنه‌های متفاوت مهاجرت داشته و لذا گرفتار جریمه‌های دسترسی شوند. لینوکس برای هر گره NUMA، یک لیست قاب آزاد جداگانه‌ای دارد، با این کار تضمین است که تخصیص حافظه‌ی یک نخ از گرهی صورت می‌گیرد که روی آن اجرا می‌شود. با تشابهی، سولاریس این مسأله را با ایجاد **lgroups** (اختصار «گروه‌های همسایگی[1]») در هسته حل می‌کند. هر lgroup پردازنده‌ها و حافظه را گرد هم می‌آورد و هر پردازنده در گروه می‌تواند هر حافظه در گروه را در محدوده‌ی یک فاصله‌ی تأخیر تعریف شده دسترسی نماید. به‌علاوه، یک سلسله‌مراتب از lgroupها مطابق میزان دیرکرد میان گروه‌ها، شبیه به سلسله‌مراتب دامنه‌های زمان‌بندی در لینوکس، وجود دارد. سولاریس تلاش می‌کند که تمامی نخ‌های یک پردازه را زمانبندی کرده و تمامی حافظه‌ی یک پردازه را از یک lgroup تخصیص بدهد. اگر این کار مقدور نباشد، برای باقی منابع مورد نیاز، lgroupهای نزدیک را بر می‌دارد. این موضوع، دیرکرد کلی حافظه را کمینه کرده و نرخ توفیق کَش پردازنده را بیشینه می‌سازد.

۱۰-۶ کوبیدگی

در نظر بگیرید که چه اتفاقی می‌افتد اگر یک پردازه قاب «کافی» نداشته باشد—یعنی، تعداد حداقل قابی که نیاز دارد تا مجموعه‌ی کاری خود را پشتیبانی کند در اختیار ندارد. پردازه خیلی سریع خطای صفحه می‌دهد. در اینجا، دنبال جایگزینی برای صفحه می‌گردد. اما، چون تمامی صفحات آن استفاده‌ی اکیدی دارند، بایستی صفحه‌ای را جایگزین کند که دوباره چند لحظه بعد استفاده خواهد داشت. در نتیجه، باز هم به سرعت خطای صفحه خواهد داد و به دفعات این قصه تکرار می‌شود، صفحاتی را برای جایگزینی انتخاب می‌کند که بلافاصله دوباره به حافظه می‌آورد.

این فعالیت شدید صفحه‌بندی، **کوبیدگی**[2] نام دارد. پردازه‌ای کوبیده می‌شود که بیشتر زمان خود را به جای اجرا صرف صفحه‌بندی نماید. همان‌گونه که انتظار دارید کوبیدگی منجر به مسئله‌های جدی کارایی می‌شود.

۱۰-۶-۱ علت کوبیدگی

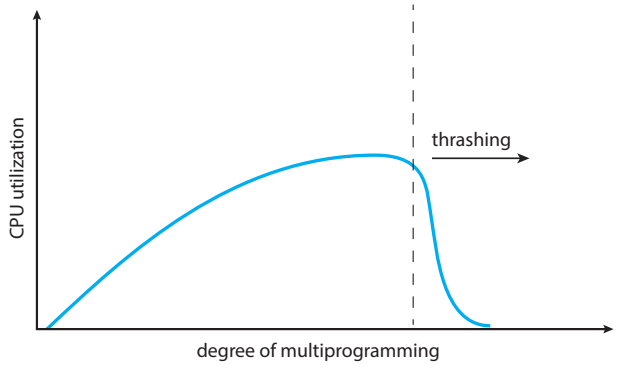
شرایط زیر را در نظر بگیرید که برگرفته از رفتار واقعی سیستم‌های صفحه‌بندی اولیه است.

1. Locality groups
2. Thrashing

سیستم‌عامل، ناظر بهره‌وری پردازنده است. اگر بهره‌وری پردازنده زیادی پایین باشد، با معرفی پردازه‌ی جدیدی به سیستم، درجه‌ی چندبرنامگی را افزایش می‌دهیم. یک الگوریتم جایگزینی صفحه‌ی سراسری استفاده می‌شود؛ صفحات را فارغ از این که به کدام پردازه تعلق دارند جایگزین می‌کند. اکنون پردازه‌ای را در نظر بگیرید که در اجرای خود وارد مرحله‌ی جدیدی می‌شود و به صفحات بیشتری نیاز پیدا می‌کند. شروع به خطا نموده و قاب‌های پردازه‌های دیگر را در اختیار می‌گیرد. این پردازه‌ها به آن صفحات نیاز پیدا می‌کنند و چون دیگر در اختیارشان نیست آنها هم خطای صفحه می‌دهند و قاب‌های دیگری را از سایر پردازه‌ها می‌گیرند. این پردازه‌های خطادهنده بایستی برای مبادله‌ی صفحات به داخل و خارج حافظه از دستگاه صفحه‌بندی استفاده کنند. همچنان که این پردازه‌ها در پشت دستگاه صفحه‌بندی به صف می‌ایستند، صف آماده خالی می‌شود. با منتظرشدن پردازه‌ها به دستگاه صفحه‌بندی، بهره‌وری پردازنده کاهش می‌یابد.

زمانبند پردازنده کاهش بهره‌وری پردازنده را می‌بیند و متعاقب آن، درجه‌ی چندبرنامگی را *افزایش* می‌دهد. برای این که پردازه‌ی تازه‌وارد راه بیفتد، قاب‌هایی را از سایر پردازه‌های در حال اجرا می‌گیرد که باعث خطای بیشتر صفحات شده و باز هم طول صف دستگاه صفحه‌بندی افزایش می‌یابد. در نتیجه، بهره‌وری پردازنده باز هم کمتر می‌شود و دوباره زمانبند پردازنده سعی می‌کند که درجه‌ی چندبرنامگی را باز هم افزایش دهد. کوبیدگی رخ می‌دهد و بازده سیستم فرو می‌نشیند. نرخ خطای صفحه به طرز فجیعی بالا می‌رود. در نتیجه، زمان دسترسی مؤثر حافظه هم افزایش می‌یابد. هیچ کار مفیدی انجام نمی‌شود، چون پردازه‌ها تمام وقت خودشان را صرف صفحه‌بندی می‌نمایند.

این پدیده در شکل ۱۰-۲۰ به تصویر کشیده می‌شود که در آن بهره‌وری پردازنده در مقایسه با درجه‌ی چندبرنامگی ترسیم شده است. با افزایش درجه‌ی چندبرنامگی، بهره‌وری پردازنده نیز هرچند به کندی، ولی افزایش می‌یابد تا این که به میزان حداکثر خود می‌رسد. اگر درجه‌ی چندبرنامگی باز هم افزایش یابد، کوبیدگی رخ می‌دهد و بهره‌وری پردازنده به شدت افت می‌کند. در اینجا، به منظور افزایش بهره‌وری پردازنده و توقف کوبیدگی بایستی درجه‌ی چندبرنامگی را *کاهش* دهیم.



شکل ۱۰-۲۰ کوبیدگی.

می‌توانیم عواقب کوبیدگی را با استفاده از یک **الگوریتم جایگزینی محلی** (یا **الگوریتم جایگزینی الویتی**) محدود کنیم. همان‌گونه که پیشتر عنوان شد، جایگزینی محلی لازم دارد که هر پردازه فقط از مجموعه‌ی قاب‌های تخصیص‌یافته به خودش انتخاب داشته باشد. بنابراین، اگر یکی از پردازه‌ها کوبیده شود، نمی‌تواند قاب پردازه‌ی دیگری را بدزدد و باعث کوبیدگی آن هم بشود. اما، مسأله هنوز به طور کامل حل نشده است. اگر پردازه‌ها کوبیده شوند، بیشتر اوقات خود را در صف انتظار دستگاه صفحه‌بندی سپری خواهند کرد. به دلیل طولانی‌ترشدن صف انتظار دستگاه صفحه‌بندی، میانگین زمان سرویس خطای صفحه افزایش خواهد یافت. بنابراین، حتی زمان دسترسی مؤثر پردازه‌ای که کوبیدگی هم ندارد، باز هم افزایش خواهد یافت.

برای پیشگیری از کوبیدگی بایستی هر تعداد قاب را که پردازه‌ها لازم دارند، فراهم کنیم. اما از کجا بدانیم که یک پردازه به چه تعداد قاب «نیاز» دارد؟ تکنیک‌های متعددی در این زمینه وجود دارند. یک راهبرد، این کار را با نظر به تعداد قاب‌های واقعی مورد استفاده توسط پردازه انجام می‌دهد. این رویکرد **مدل همسایگی**[1] اجرای پردازه را تعریف می‌کند.

تعبیر مدل همسایگی این است که پردازه با اجرای خود از محلی به محل دیگر جابجا می‌شود. همسایگی، مجموعه‌ای از صفحات است که به طور اکید با همدیگر استفاده می‌شوند. به طور کلی، یک برنامه‌ی در حال اجرا مرکب از چندین همسایگی متفاوت است که امکان هم‌پوشانی هم دارند. برای مثال، لحظه‌ای که یک تابع فراخوانی می‌شود، همسایگی جدیدی تعریف می‌کند. این همسایگی شامل مراجعات حافظه‌ی دستورات تابع، متغیرهای محلی و زیرمجموعه‌ای از متغیرهای سراسری است. زمانی که از این تابع خارج می‌شویم، پردازه این همسایگی را ترک می‌کند، چرا که دیگر متغیرهای محلی و دستورات تابع استفاده ندارند. ممکن است باز هم به این همسایگی برگردیم.

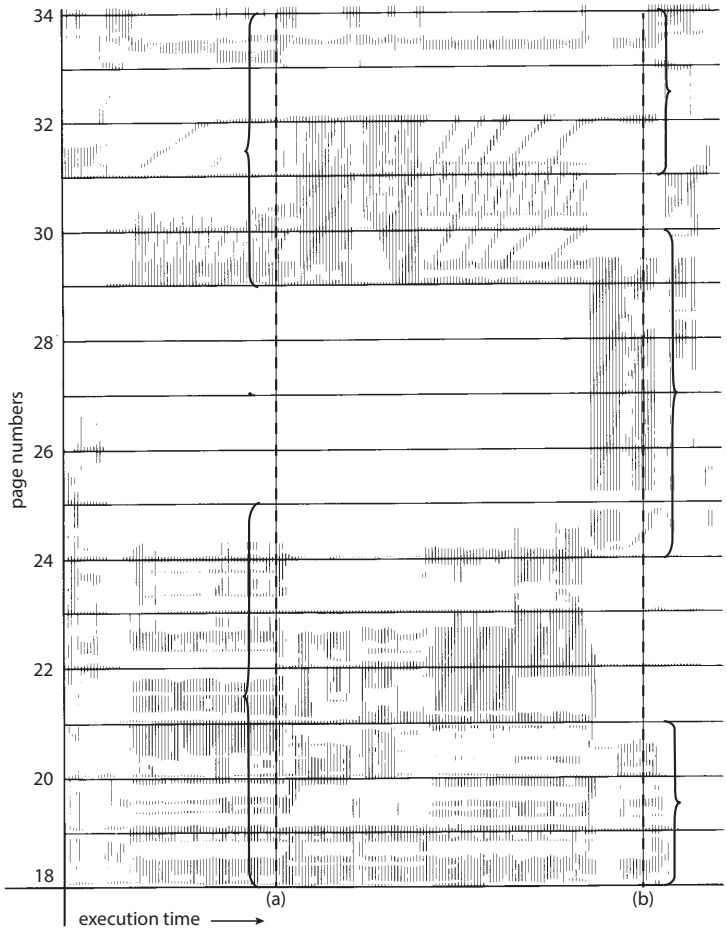
شکل ۱۰-۲۱، مفهوم همسایگی و چگونگی تغییر همسایگی یک پردازه را به‌مرور زمان نمایش می‌دهد. در لحظه‌ی (a)، همسایگی، مجموعه‌ی صفحات {18, 19, 20, 21, 22, 23, 24, 29, 30, 33} می‌باشد. در لحظه‌ی (b)، همسایگی به {18, 19, 20, 24, 25, 26, 27, 28, 29, 31, 32, 33} تغییر می‌کند. به همپوشانی توجه کنید که برخی از صفحات (برای مثال، ۱۸، ۱۹، و ۲۰) در هر دو همسایگی هستند.

بنابراین، می‌بینیم که همسایگی‌ها با ساختار برنامه و ساختمان‌داده‌های آن تعریف می‌شوند. تعبیر مدل همسایگی آن است که تمامی برنامه‌ها این ساختار اساسی مراجعات حافظه را به نمایش خواهند گذاشت. توجه کنید که مدل همسایگی اصل ناگفته‌ای است که تاکنون در این کتاب در پس مباحث کَش مطرح بوده است. اگر دسترسی هر یک از انواع داده‌ها به جای الگوی مشخص به صورت تصادفی انجام شود، نهان‌کردن فایده‌ای نخواهد داشت.

فرض کنید که قاب کافی مورد نیاز پوشش همسایگی فعلی یک پردازه را تخصیص بدهیم. این پردازه برای صفحاتی که در همسایگی آن قرار دارند خطای صفحه می‌دهد تا این که همگی آن صفحات را داخل حافظه می‌بیند؛ پس از آن، دیگر خطای صفحه نمی‌گیرد تا این که همسایگی خود

1. Locality model

را تغییر بدهد. اگر تعداد قاب‌های تخصیصی پردازه از اندازه‌ی همسایگی فعلی آن کمتر باشند، چون پردازه نمی‌تواند تمامی صفحات همسایگی خود را برای استفاده در حافظه نگه بدارد، کوبیده خواهد شد.



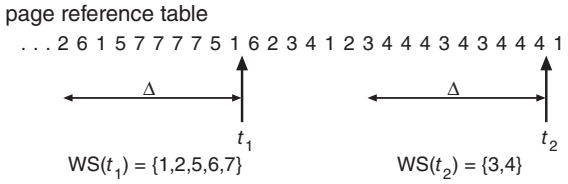
شکل ۱۰-۲۱ همسایگی در یک الگوی مراجعه به حافظه.

۱۰-۶-۲ مدل مجموعه‌ی کاری

همان‌گونه که متذکر شدیم، **مدل مجموعه‌ی کاری** بر اساس فرض همسایگی است. این مدل از یک پارامتر Δ برای تعریف **پنجره‌ی مجموعه‌ی کاری** استفاده می‌کند. ایده آن است که اخیرترین Δ مراجعات صفحه را در نظر بگیریم. مجموعه‌ی صفحاتی که در اخیرترین Δ مراجعات صفحه قرار می‌گیرند، **مجموعه‌ی کاری** را تشکیل می‌دهند (شکل ۱۰-۲۲). اگر صفحه‌ای مؤکد استفاده شود، در مجموعه‌ی کاری قرار خواهد گرفت. در صورتی که از صفحه استفاده نشود، با

گذشت Δ واحد زمانی از آخرین بار مراجعه‌اش از مجموعه‌ی کاری بیرون می‌افتد. بنابراین، مجموعه‌ی کاری تقریبی از همسایگی برنامه است.

برای مثال، با داشتن دنباله‌ی مراجعاتی که در شکل ۱۰-۲۲ نشان داده می‌شوند، اگر $\Delta = ۱۰$ مراجعه به حافظه باشد، در این صورت مجموعه‌ی کاری لحظه‌ی t_1 برابر $\{1,2,5,6,7\}$ خواهد بود. مجموعه‌ی کاری لحظه‌ی t_2 هم به $\{3,4\}$ تغییر می‌یابد.



شکل ۱۰-۲۲ مدل مجموعه‌ی کاری.

صحت مجموعه‌ی کاری به انتخاب Δ بستگی دارد. اگر Δ زیادی کوچک باشد، نمی‌تواند کل همسایگی را بپوشاند؛ اگر Δ خیلی هم بزرگ باشد، امکان دارد چندین همسایگی را روی هم بیندازد. در حد نهایت آن، اگر Δ نامحدود باشد، مجموعه‌ی کاری، مجموعه‌ی صفحاتی است که پردازه در طول اجرای خود با آنها مواجه می‌شود.

پس، مهمترین ویژگی مجموعه‌ی کاری، اندازه‌ی آن است. اگر اندازه‌ی مجموعه‌ی کاری هر یک از پردازه‌های سیستم، WSS_iها را محاسبه کنیم، رابطه‌ی زیر را بدست می‌آوریم:

$$D = \sum WSS_i$$

که D تعداد کل قاب‌های مورد نیاز پردازه‌هاست. هر پردازه به صورت مؤکد، صفحات مجموعه‌ی کاری خود را استفاده می‌کند. بنابراین، پردازه‌ی i به WSS_i قاب نیاز دارد. اگر تعداد کل قاب‌های مورد نیاز از تعداد کل قاب‌های موجود بزرگ‌تر باشد ($D > m$)، کوبیدگی اتفاق خواهد داد، چون برخی از پردازه‌ها قاب کافی در اختیار نخواهند داشت.

همین که Δ انتخاب شود، استفاده از مدل مجموعه‌ی کاری ساده است. سیستم‌عامل، ناظر مجموعه‌ی کاری هر یک از پردازه‌ها بوده و مطابق اندازه‌ی مجموعه‌ی کاری هر یک از پردازه‌ها، قاب‌های کافی را در اختیار پردازه‌ها قرار می‌دهد. در صورتی که قاب‌های اضافی کافی موجود باشد، پردازه‌ی دیگری می‌تواند آغاز شود. اگر مجموع اندازه‌های مجموعه‌های کاری از تعداد کل قاب‌های موجود تجاوز نماید، سیستم‌عامل پردازه‌ای را انتخاب کرده و معلق می‌سازد. صفحات پردازه‌ی معلق در دیسک نوشته می‌شوند (مبادله به بیرون) و قاب‌های آن در اختیار سایر پردازه‌ها قرار می‌گیرند. پردازه‌ی معلق بعداً می‌تواند دومرتبه از سر گرفته شود.

این راهبرد مجموعه‌ی کاری از کوبیدگی پیشگیری کرده و درجه‌ی چندبرنامگی را تا حد ممکن بالا نگه می‌دارد. بنابراین، بهره‌وری پردازنده را بهینه می‌سازد. مشکل مدل مجموعه‌ی کاری حفظ رد مجموعه‌ی کاری است. پنجره‌ی مجموعه‌ی کاری، پنجره‌ای متحرک است. در هر مراجعه به حافظه، مراجعه‌ی جدیدی در انتهای پنجره ظاهر می‌شود و از سویی دیگر، قدیمی‌ترین مراجعه

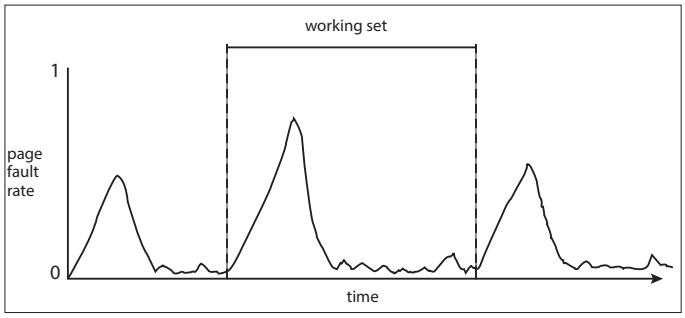
بیرون می‌افتد. یک صفحه در مجموعه‌ی کاری است، اگر در مکانی از پنجره‌ی مجموعه‌ی کاری مراجعه شده باشد.

می‌توانیم مدل مجموعه‌ی کاری را با یک تایمر که در فواصل زمانی ثابتی وقفه صادر می‌کند و یک بیت مراجعه، تقریب بزنیم. برای مثال، فرض کنید Δ برابر ۱۰۰۰۰ مراجعه باشد و تایمر برای هر ۵۰۰۰ بار مراجعه یک وقفه تولید کند. زمانی که یک وقفه از تایمر می‌گیریم، مقادیر بیت‌های مراجعه‌ی هر یک از صفحات را کپی کرده و پاک می‌کنیم. بنابراین، اگر خطای صفحه‌ای اتفاق بیفتد، با بررسی بیت مراجعه‌ی فعلی و دو بیت حافظه می‌توانیم تعیین کنیم که آیا یک صفحه در ۱۰۰۰۰ تا ۱۵۰۰۰ مراجعه‌ی اخیر استفاده داشته است یا خیر. اگر صفحه مراجعه داشته باشد، حداقل یکی از این بیت‌ها یک خواهند بود. اگر صفحه استفاده نشده باشد، این بیت‌ها صفر خواهند بود. آن دسته از صفحاتی که حداقل یکی از این سه بیت آنها یک بوده باشد در مجموعه‌ی کاری قرار می‌گیرند.

توجه کنید که این ترتیب نیز صحت کاملی ندارد، چون نمی‌توان گفت که مراجعه به صفحه در کجای این فاصله‌ی ۵۰۰۰ مراجعه صورت گرفته است. برای کاستن این عدم قطعیت می‌توانیم بیت‌های سابقه و فرکانس وقفه‌ها را افزایش دهیم (برای مثال، ۱۰ بیت و یک وقفه برای هر ۱۰۰۰ بار مراجعه). با این وجود، هزینه‌ی سرویس‌دهی این تعداد وقفه هم متعاقب آن بیشتر خواهد شد.

مجموعه‌های کاری و نرخ خطای صفحات

رابطه‌ی مستقیمی میان مجموعه‌ی کاری هر پردازه و نرخ خطای صفحه‌ی آن وجود دارد. همان‌گونه که در شکل ۱۰-۲۲ نشان داده می‌شود، مجموعه‌ی کاری هر پردازه نوعاً با گذر زمان و با جابجایی مراجعات بخش‌های کد و داده‌ها از یک همسایگی به همسایگی دیگر تغییر می‌یابد. با فرض این که حافظه‌ی کافی برای ذخیره‌سازی مجموعه‌ی صفحات کاری پردازه موجود باشد (یعنی، پردازه کوبیده نشود)، نرخ خطای صفحه با گذشت زمان، بین قله‌ها و دره‌ها را طی خواهد کرد. این رفتار کلی در زیر نشان داده می‌شود:



زمانی یک نقطه‌ی اوج در نرخ خطای صفحه پیش می‌آید که صفحه‌بندی نیازی را در یک همسایگی جدید آغاز می‌کنیم. اما، همین که مجموعه‌ی کاری این همسایگی جدید در حافظه قرار می‌گیرد، نرخ خطای صفحه افت می‌کند. زمانی که پردازه به همسایگی جدیدی حرکت می‌کند، نرخ خطای صفحه باز هم به سمت یک قله بالا رفته، سپس با بارگذاری مجموعه‌ی کاری جدید در درون حافظه، به نرخ پایین‌تری بر می‌گردد. فاصله‌ی

زمانی میان آغاز یک قله و شروع قله‌ی بعدی، انتقال از یک مجموعه‌ی کاری به مجموعه‌ی کاری دیگر را نشان می‌دهد.

۱۰-۶-۳ فرکانس خطای صفحه

مدل مجموعه‌ی کاری موفقیت‌آمیز است و دانش مجموعه‌ی کاری هم می‌تواند برای پیش‌بند صفحه (بخش ۱۰-۹-۱) مفید باشد، اما به نظر می‌رسد که برای کنترل کوبیدگی، روش پخته‌ای نیست. یک راهبرد که از **فرکانس خطای صفحه**[1] **(PFF)** استفاده می‌کند رویکرد مستقیمی در پیش می‌گیرد.

مسأله‌ی به‌خصوص آن است که چگونه از کوبیدگی پیشگیری کنیم. کوبیدگی نرخ خطای صفحه‌ی بالایی دارد. بنابراین، می‌خواهیم نرخ خطای صفحه را کنترل کنیم. موقعی که خیلی بالاست، می‌دانیم که پردازه به قاب‌های بیشتری نیاز دارد. برعکس، اگر نرخ خطای صفحه زیادی پایین باشد، در اینجا ممکن است پردازه قاب‌های بسیار زیادی در اختیار داشته باشد. می‌توانیم روی نرخ خطای مطلوب، یک کران بالا و پایین بگذاریم (شکل ۱۰-۲۳). اگر نرخ خطای صفحه از حد بالا تجاوز نماید، قاب دیگری به پردازه تخصیص می‌دهیم. اگر نرخ خطای صفحه به زیر حد پایین‌تر برسد، یک قاب از پردازه بر می‌داریم. بنابراین، برای پیشگیری از کوبیدگی می‌توانیم نرخ خطای صفحه را به طور مستقیم سنجیده و کنترل کنیم.

همان‌گونه که در راهبرد مجموعه‌ی کاری داشتیم، ممکن است به ناچار پردازه‌ای را به خارج از حافظه مبادله کنیم. اگر نرخ خطای صفحه افزایش یابد و هیچ قاب آزادی نباشد، بایستی پردازه‌ای را انتخاب کرده و به ذخیره‌ی پشتیبان مبادله کنیم. در ادامه، قاب‌های آزاد شده در میان پردازه‌هایی که نرخ خطای صفحه‌ی بالایی دارند، توزیع می‌شوند.

۱۰-۶-۴ مشق فعلی

به بیان عملی، کوبیدگی و مبادله‌ی حاصل به‌واقع یک پیامد ناخوشایند گرانی بر کارایی دارند. بهترین مشق[2] فعلی در پیاده‌سازی یک سیستم کامپیوتری این است که هر جا مقدور باشد، برای اجتناب از کوبیدگی و مبادله، حافظه‌ی فیزیکی کافی لحاظ شود. از گوشی‌های هوشمند گرفته تا کارگزارهای بزرگ، تدارک حافظه‌ی کافی برای حفظ تمامی مجموعه‌ی کاری به صورت همروند در حافظه، مگر در شرایط استثناء، بهترین تجربه را برای کاربر فراهم می‌کند.

۱۰-۷ فشرده‌سازی حافظه

یک جایگزین برای صفحه‌بندی، **فشرده‌سازی حافظه** است. در اینجا، به‌جای این که قاب‌های تغییریافته به فضای مبادله برده شوند، چندین قاب را به‌شکل یک قاب منفرد فشرده می‌کنیم، این کار سیستم را قادر می‌سازد تا استفاده از حافظه را کاهش بدهد بدون این که به مبادله‌ی صفحات اقدام کند.

1. Page-fault frequency
2. Practice